

Sienovo Marine · 硬件接入文档

Sienovo Marine · 硬件接入文档

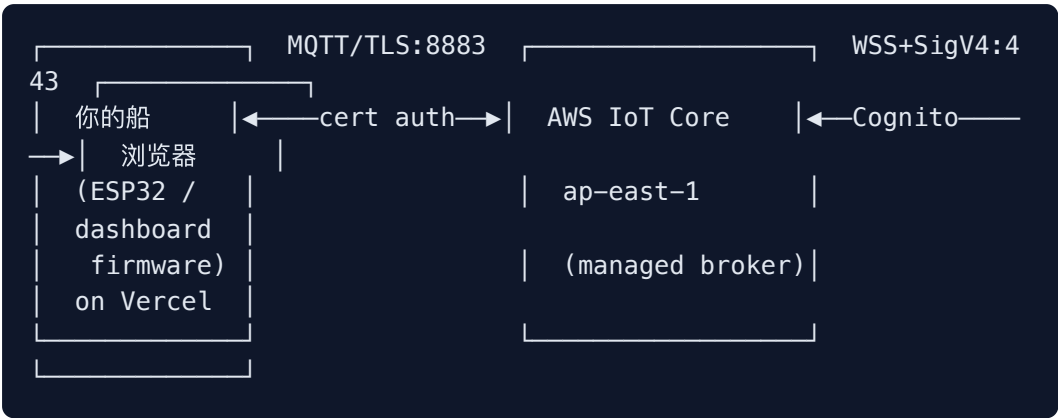
1. 系统架构
2. 接入凭证
3. 连接参数
4. Topic 协议
5. Payload 格式
6. 行为约定
7. 自测流程
8. ESP-IDF 接入示例 (C 伪代码)
9. 常见坑
10. 不在你范围内的事
11. 联络 / 回报

Sienovo Marine · 硬件接入文档

面向船端固件团队 (ESP32 / 树莓派 / 其它 MCU)。读完这份文档你应该能：

1. 让一台真实船硬件作为 IoT thing 接入云端
 2. 上行遥测/事件/摄像头帧
 3. 下行接收并执行控制命令
 4. 对照参考实现 `simulator/boat.mjs` 自检
-

1. 系统架构



云端是托管的，不需要你部署任何服务。你只负责”船”那一端——按本文协议接入即可。

2. 接入凭证

每艘船需要四样东西，由云端运维提供：

文件	作用	大小
<code>BOAT-XXXX.cert.pem</code>	X.509 客户端证书	~1.2 KB
<code>BOAT-XXXX.private.key</code>	证书私钥（永不外传）	~1.7 KB
<code>AmazonRootCA1.pem</code>	验证 broker 端身份的根 CA	~1.2 KB
<code>BOAT-XXXX</code> （字符串）	thing name = MQTT client ID = 在 topic 里的 boat ID	—

怎么拿到：联系云端运维同学，提供你要的 thing name（如 `BOAT-XXXX`），运维会回你三个文件（`*.cert.pem` / `*.private.key` / `AmazonRootCA1.pem`）。
密钥怎么生成不在你这边的范围，烧入硬件即可。

烧入位置建议： – 三个 `.pem` 文件烧到 SPIFFS / LittleFS，或者 ESP-IDF 的 `nvs partition`；私钥分区禁止可读 dump – 量产时每台船一张唯一证书；不要多台共用同一证书（policy 校验会失败）

3. 连接参数

项	值
Endpoint (broker host)	a36ytt8gq852xf-ats.iot.ap-east-1.amazonaws.com
Port	8883 (MQTT over TLS)
Protocol	MQTT 3.1.1 (推荐) 或 5.0
Authentication	mTLS · 客户端 X.509 证书 (由 broker 校验)
TLS server cert verification	启用, 使用上面的 AmazonRootCA1.pem
MQTT Client ID	必须等于 thing name (如 B0AT-A3F8, 区分大小写)
Keep-alive	60 秒推荐
Clean session	true (v1, 等后续我们做持久会话再改)

关键约束:

-  **NTP 时间必须同步**: TLS 证书校验对时间敏感, 开机必须先校时 (任意公网 NTP 都行, 国内可用 ntp.aliyun.com)。否则握手永远失败、报 bad certificate 之类的错
-  **禁止把 client ID 写成别的**: 必须 = thing name, 否则 IoT policy 中 \${iot:Connection.Thing.ThingName} 解析不出, 所有 publish/subscribe 都会被拒
-  **断线后重连**: 建议指数退避 (1s, 2s, 4s, 最多 30s), 不要硬循环

4. Topic 协议

所有 topic 形如 sienovo/boats/{boatId}/{kind}:

上行 (boat → cloud)，你只能发到自己 boatId 的 topic

Topic	频率	用途
sienovo/boats/{id}/state	5 Hz (每 200ms)	实时遥测，整个状态全发
sienovo/boats/{id}/event	按需	单次事件 (投饵完成、到达航点、告警等)
sienovo/boats/{id}/camera	1-2 Hz	摄像头帧 (base64 image data URL)

下行 (cloud → boat)，你订阅这一条

Topic	用途
sienovo/boats/{id}/control	控制命令 (油门/方向/投饵/补光/返航/故障注入)

QoS 选择：全部用 **QoS 0** (fire-and-forget)。状态消息丢一两条没影响，下一条就刷新；控制命令也是高频幂等。要做 QoS 1 等量产稳定后再升级。

5. Payload 格式

格式声明 (v1)：MQTT 协议本身只传字节流，payload 可以是任何格式。**v1 全部 topic 使用 UTF-8 编码的 JSON 文本——便于调试和迭代。**

未来可能变化 (优先级高到低)：

- camera topic 改为**直接发送 raw JPEG 二进制字节** (省 33% base64 膨胀 + 省 JSON 解析)，最可能优先升级
- state topic 高频部分 (位置/速度等) 改为 **Protocol Buffers** 或 **MessagePack**，体积压到 1/3
- event 和 control 大概率长期保持 JSON (频率低、字段灵活)

硬件组建议：把序列化 / 反序列化封装成单独函数，未来切换格式只改这一层，业务代码不用动。任何协议变更都会在本文 PR 通知。

5.1 上行 · `sienovo/boats/{id}/state` (JSON)

```
{
  "id": "BOAT-A3F8",
  "lat": 30.27410,
  "lng": 120.15510,
  "heading": 18,
  "speed": 0.83,
  "battery": 87.5,
  "signal": 91,
  "baitLevel": 100,
  "distance": 145,
  "depth": 4.2,
  "waterTemp": 17.3,
  "ir": false,
  "light": false,
  "isOnline": true,
  "components": {
    "motor": "normal",
    "battery": "normal",
    "gps": "normal",
    "camera": "normal",
    "light": "normal",
    "baitDispenser": "normal",
    "rudder": "normal",
    "link": "normal"
  }
}
```

字段说明：

字段	单位 / 取值	说明
<code>id</code>	string	thing name, 必须和 client ID 一致
<code>lat</code> / <code>lng</code>	度 (WGS84)	GPS 经纬度, 6 位小数 足够 (~10cm 精度)
<code>heading</code>	度, 0-360	0=正北, 顺时针
<code>speed</code>	m/s	当前实际速度 (不是设定值)
<code>battery</code>	0-100	电池电量百分比
<code>signal</code>	0-100	4G / 网络信号强度抽象值
<code>baitLevel</code>	0-100	饵料仓剩余百分比
<code>distance</code>	米	距 home 点的直线距离
<code>depth</code>	米	当前水深 (声呐读数)
<code>waterTemp</code>	摄氏度	水温
<code>ir</code>	bool	红外摄像头是否开启
<code>light</code>	bool	补光灯是否开启
<code>isOnline</code>	bool	必填 <code>true</code> (broker 用心跳判离线, 但你也得显式发)
<code>components.*</code>	<code>"normal"</code> / <code>"warning"</code> / <code>"damaged"</code> / <code>"offline"</code>	各配件健康状态, 由你自检

注意: – 整个对象每帧全发, 不要做增量 (dashboard 是无状态订阅者) – 浮点保留 2-6 位小数即可, 没必要塞到 17 位 – 单帧大小目标 < 600 字节 (保守 4G 流量)

5.2 上行 · `sienovo/boats/{id}/event` (JSON)

按需触发的”一次性事件”——区别于 `state` (连续遥测, 覆盖式):

判断 **state vs event** 的窍门 – 这条数据”丢一帧没事，下一帧覆盖就行”？ → 放 **state** – 这条数据”是一次发生，错过就丢失意义”（投饵完成、到达航点、告警）？ → 放 **event**

type 是必填字段；其它字段按事件类型自定义。下面是v1 协议覆盖的全部标准事件类型——硬件组按需发送，dashboard 端会渲染对应 UI。

5.2.1 投饵流程事件

```
// 用户在 dashboard 点"投饵", 命令到达船端 → 你打开闸门那一瞬间
{ "type": "bait-released", "remaining": 80, "lat": 30.27410, "lng": 120.15510, "weight_g": 50 }

// 饵料仓清空 (baitLevel 跨过 0)
{ "type": "bait-empty" }

// 接到 bait-at-waypoint 命令, 开始去窝点
{ "type": "going-to-waypoint", "lat": 30.276, "lng": 120.157, "wp_id": 3 }

// 到达窝点开始投放
{ "type": "arrived-at-waypoint", "wp_id": 3 }

// 自动定点打窝模式下完成一轮所有窝点
{ "type": "auto-bait-complete", "rounds": 5, "total_g": 250 }
```

字段约定： – **remaining** — 0-100，投后剩余百分比（投饵后必发） – **lat** / **lng** — 投放时的实际 GPS 坐标（非命令坐标，能差 1-2 米都正常） – **weight_g** — 本次实际投放克数（如有称重传感器；没有就别加这字段） – **wp_id** — 航点编号（dashboard 给你的 set-waypoint 不带 ID，你自己排序加）

5.2.2 自主航行事件

```
// 接到 return-home 命令, 开始返航
{ "type": "returning-home" }

// 已到 home 点
{ "type": "arrived-home" }

// 接到 set-waypoint 命令并加入队列
{ "type": "waypoint-added", "wp_id": 3, "queue_size": 4 }

// 接到 clear-waypoints
{ "type": "waypoints-cleared" }
```

5.2.3 告警事件

任何告警都用 `type: "alert"`，下面是 `code` 取值表：

```
{ "type": "alert", "code": "low-battery",      "level": "warning", "battery": 18 }
{ "type": "alert", "code": "critical-battery", "level": "damaged", "battery": 4 }
{ "type": "alert", "code": "gps-lost",        "level": "warning" }
{ "type": "alert", "code": "gps-recovered",    "level": "info" }
{ "type": "alert", "code": "link-degraded",    "level": "warning", "signal": 25 }
{ "type": "alert", "code": "motor-overcurrent", "level": "damaged", "amps": 12.3 }
{ "type": "alert", "code": "motor-overheat",   "level": "warning", "temp_c": 78 }
{ "type": "alert", "code": "collision",        "level": "damaged", "axis": "front" }
{ "type": "alert", "code": "water-ingress",    "level": "damaged", "d" }
{ "type": "alert", "code": "geofence-exit",    "level": "warning", "lat": 30.28, "lng": 120.17 }
```

level 取值：`info` / `warning` / `damaged`（与 `components.*.status` 同一套语义）。

节流原则：同一个 alert code 不要 1Hz 狂发；建议节流到每 30 秒最多一次，状态恢复时发对应的 `*-recovered` 事件即可。

5.2.4 维护 / 生命周期事件

```
// 开机自检完成
{ "type": "boot-complete", "fw_version": "v1.2.3", "uptime_s": 0
  }

// 即将关机（用户长按 / 远程 shutdown）
{ "type": "shutting-down", "reason": "user" }

// 配件刚做了维护更换
{ "type": "maintenance", "component": "baitDispenser", "action": "refilled", "by": "field-tech-A" }
```

5.2.5 完整投饵案例（以 dashboard 点”投饵”为起点）

时间线，全部由船端发出：

```
T=0ms      收到 control: { type: "release-bait" }
T=10ms     打开闸门
T=200ms    闸门关闭, 称重 -50g
T=210ms    PUBLISH event: { "type": "bait-released", "remaining":
80, "lat": ..., "lng": ..., "weight_g": 50 }
T=210ms    下一个 state 帧带上 baitLevel=80 (自然刷新)
```

如果是 dashboard 点“定点打窝” (bait-at-waypoint):

```
T=0        收到 control: { type: "bait-at-waypoint", lat, lng }
T=10ms     PUBLISH event: { "type": "going-to-waypoint", lat, lng,
wp_id: 7 }
... 自主导航中 (state 持续刷新位置) ...
T=14s      到达
T=14s      PUBLISH event: { "type": "arrived-at-waypoint", wp_id:
7 }
T=14.2s    投饵完成
T=14.2s    PUBLISH event: { "type": "bait-released", remaining: 6
0, ..., weight_g: 50 }
```

5.3 上行 · sienovo/boats/{id}/camera (JSON)

重要架构原则: MQTT 上的 camera topic 只用来传“缩略图 / 预览帧”——给 dashboard 看个画面变化、确认船在动就行。真正的实时视频流不走 MQTT, 走 WebRTC (详见 §5.6)。

5.3.1 Payload 格式

```
{
  "frame": "data:image/jpeg;base64,/9j/4AAQSkZJRgABAQ...",
  "ts": 1712345678901,
  "w": 320,
  "h": 240
}
```

字段	类型	说明
<code>frame</code>	string	image data URL, 支持 <code>image/jpeg</code> 或 <code>image/png</code> (强烈推荐 JPEG, 压缩比好 5-10 倍)
<code>ts</code>	number	帧时间戳 (毫秒), 用于乱序检测
<code>w / h</code>	number	可选; 帧的宽高 (dashboard 用来画占位框, 没有也能显示)

5.3.2 硬性参数 (超过会被 broker 截断或拒绝)

项	值	为什么
单帧最大	120 KB	AWS IoT 单条消息硬上限 128KB; 扣掉 base64 膨胀 33% + JSON 包裹
推荐分辨率	320×240 或 640×360	任何 720p 以上的图都会撞上限
JPEG 质量	40-60	60 已经够看清水面 / 鱼漂; 不需要更高
频率	1-2 Hz	高了消息计费爆掉; 做实时取景请用 WebRTC
单帧理想大小	5-15 KB	4G 上行带宽 ~2 Mbps, 留足 state / 控制余量

5.3.3 ESP32 + OV2640/OV5640 实现要点

ESP32 用官方 `esp_camera` 库:

```

camera_config_t cam = {
    .pixel_format = PIXFORMAT_JPEG,    // 摄像头硬件直出 JPEG, 不用 C
    PU 编码
    .frame_size   = FRAMESIZE_QVGA,    // 320x240
    .jpeg_quality = 12,                 // 0-63, 越小质量越高; 12 ≈
    JPEG quality 60
    .fb_count     = 2,                 // 双 buffer 防丢帧
    // ... 引脚配置略
};
esp_camera_init(&cam);

// 每 500ms 抓帧 + 推送
camera_fb_t* fb = esp_camera_fb_get();
if (fb && fb->len < 30 * 1024) { // 限大小, 过大丢
    // base64 编码 → 拼成 data URL → MQTT publish
    char* b64 = base64_encode(fb->buf, fb->len);
    char topic[64], payload[fb->len * 2];
    snprintf(topic, sizeof(topic), "sienovo/boats/%s/camera", THING_NAME);
    snprintf(payload, sizeof(payload),
        "{\"frame\":\"data:image/jpeg;base64,%s\",\"ts\":%lld,\"w\":"
        "\"320\",\"h\":\"240}\"",
        b64, esp_timer_get_time() / 1000);
    esp_mqtt_client_publish(client, topic, payload, 0, 0, 0);
    free(b64);
}
esp_camera_fb_return(fb);

```

5.3.4 自适应：网络差时降级

伪代码：

```

static int target_fps = 2;
static int target_quality = 12;

// 监控 publish 队列长度 / 4G RSSI / 上次 publish 是否超时
void on_network_status(int rssi, int queue_depth) {
    if (queue_depth > 5 || rssi < -100) {
        target_fps = 1;        // 降到 1Hz
        target_quality = 25;    // 质量降一档 (更小的帧)
    } else if (rssi > -85 && queue_depth < 2) {
        target_fps = 2;        // 恢复
        target_quality = 12;
    }
}

```

最坏情况：完全停止 camera publish, 让 state 继续推——船的位置和遥测比画面重要得多。

5.3.5 红外 / 补光的影响

收到 `set-camera-mode` 后：

信号	硬件动作
<code>ir: true</code>	切到红外感光模式（OV2640 没有原生 IR，需要外接红外滤光片机械切换 + 红外 LED 阵列点亮）
<code>light: true</code>	主补光 LED 阵列点亮（注意散热 + 电流，可能需要降低 PWM）

切换后下一帧 `state` 必须把 `state.ir` / `state.light` 更新成实际状态——不要假设命令一定生效（LED 烧了、机械切换卡了都可能失败）。失败了 → 推 `event { "type": "alert", "code": "camera-mode-switch-failed" }`。

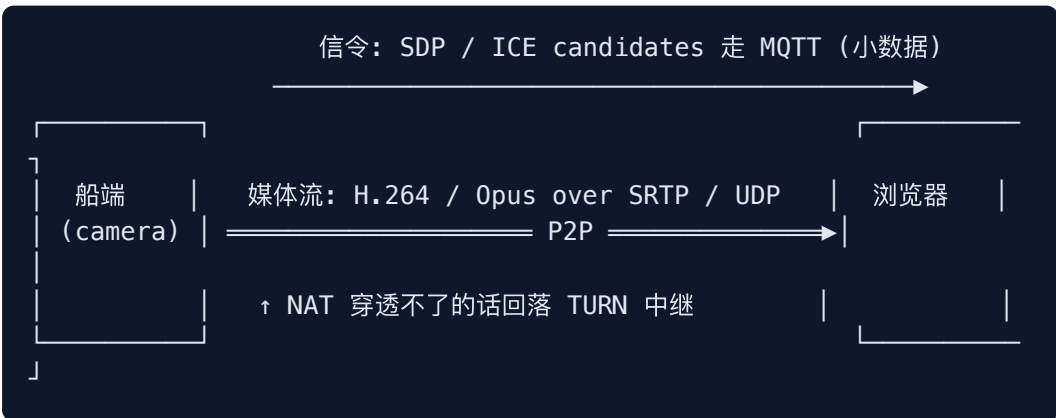
5.4 实时视频 / WebRTC（v1.5+ 路线，不在当前协议）

告知性章节——v1 不实现。下面只是让硬件组提前知道架构走向，物料选型时考虑兼容。

走 WebRTC 而不是 MQTT，原因：

- **延迟**：WebRTC P2P 30–100ms vs MQTT through cloud 150–300ms（远程驾驶感差异巨大）
- **带宽**：H.264 实时视频 1–3 Mbps，MQTT 单条 128KB 上限根本扛不住
- **成本**：MQTT 按消息计费，30fps 视频每船每天 260 万条消息直接破产

架构：



候选实现： 1. **AWS Kinesis Video Streams WebRTC**（推荐）—— 自带 STUN+TURN，浏览器有官方 SDK；ESP32 端可用 [amazon-kinesis-video-streams-webrtc-sdk-c](#) 2. **LiveKit / Daily / mediasoup** —— 第三方托管，简单但脱离 AWS 栈 3. **RTMP 推流 + HLS 拉流** —— 一对多分发好，但延迟 5-10 秒，不适合驾驶

硬件选型预留：摄像头模组优先选硬件 H.264 编码款（如 OV5640 + ESP32-S3 编码器、或专用 ISP），不要只支持 JPEG——v1.5 上 WebRTC 时直出 H.264 才不会让 CPU 拖崩。

5.5 下行 · `sienovo/boats/{id}/control` (JSON)

你订阅这条 topic，对每条收到的 message 解析并执行：

```
{ "type": "control", "command": { "throttle": 0.8, "rudder": -0.3 } }
{ "type": "set-camera-mode", "ir": true, "light": false }
{ "type": "release-bait" }
{ "type": "return-home" }
{ "type": "set-waypoint", "lat": 30.276, "lng": 120.157 }
{ "type": "clear-waypoints" }
{ "type": "bait-at-waypoint", "lat": 30.276, "lng": 120.157 }
{ "type": "set-fault", "component": "motor", "status": "warning" }
```

字段说明：

type	字段	说明
control	<code>command.throttle</code> $\in [-1, 1]$ (负数=倒车, 半速) <code>command.rudder</code> $\in [-1, 1]$ (负=左, 正=右)	主驾驶。指令是 绝对值 , 不是增量
set-camera-mode	<code>ir?</code> 、 <code>light?</code> (任一可选 boolean)	切红外 / 补光灯开关
release-bait	—	立即开闸投一次饵料 (典型扣 20% <code>baitLevel</code>)
return-home	—	自主返航
set-waypoint	<code>lat</code> 、 <code>lng</code>	加入航点队列 (队列由你管)
clear-waypoints	—	清空航点队列
bait-at-waypoint	<code>lat</code> 、 <code>lng</code>	立即去某点投饵
set-fault	<code>component</code> (<code>key</code> $\in$ <code>components</code> 字段名)、 <code>status</code> (normal/warning/damaged)	模拟器/调试用; 真实船端可忽略或视作”维护模式标志”

安全约定： – 接到 `control` 命令后, 如果 1 秒内没收到下一条, **自动归零** (防遥控信号丢失船开着不停)。模拟器目前没做这个, 但**真船必须做** – `throttle` 的限速、`rudder` 的转向限速都按硬件能力裁剪, 不要直接照搬 – 收到自己看不懂的 `type` : 忽略并记日志, 不要崩溃

6. 行为约定

项	期望行为
开机	NTP 同步 → 加载证书 → 连 broker → 订阅 control → 开始 5Hz 推 state
网络断开	不抛异常，指数退避重连；本地继续维持上一次 control 指令 1 秒后归零
electricity drop / 低电	<code>components.battery</code> 标 warning (<20%) / damaged (<5%)； <code>battery</code> 字段照实报
GPS 丢失	<code>components.gps</code> 标 warning； <code>lat/lng</code> 报最后一次有效值
执行返航	推 event 类型 <code>returning-home</code> ； 到达后推 <code>arrived-home</code>
关机	发一条 <code>state</code> 标 <code>isOnline: false</code> ，再优雅断开 MQTT

7. 自测流程

1. 用我们的 dashboard 当观察方：浏览器打开 `https://<部署 url>/boats/BOAT-XXXX/status`
2. 跑你的固件，配好证书 + thing name = `BOAT-XXXX`
3. 看连接状态从”等待船上线” → “在线”
4. 看遥测数字开始跳动；摄像头格出现你的画面
5. 测下行：dashboard 上拖油门、按方向键 → 你的固件应该收到 control 消息并体现在硬件动作上
6. 故障注入测试：dashboard 故障注入下拉选“电机损坏” → 你的下一帧 state 里 `components.motor` 应该是 `"damaged"`

参考实现：[simulator/boat.mjs](#)。它就是 Node.js 版本的”船”，整套协议都跑得起来。可以直接拿来对照。

8. ESP-IDF 接入示例 (C 伪代码)

```
#include "mqtt_client.h"
#include "esp_tls.h"
#include "cJSON.h"

extern const uint8_t cert_pem_start[] asm("_binary_BOAT_A3F8_certificate_pem_start");
extern const uint8_t key_pem_start[] asm("_binary_BOAT_A3F8_private_key_start");
extern const uint8_t root_ca_pem_start[] asm("_binary_AmazonRootCA1_pem_start");

static const char* THING_NAME = "BOAT-A3F8";

static esp_mqtt_client_handle_t client;

static void on_event(void* arg, esp_event_base_t base, int32_t event_id, void* event_data) {
    esp_mqtt_event_handle_t evt = event_data;
    switch ((esp_mqtt_event_id_t)event_id) {
        case MQTT_EVENT_CONNECTED:
            esp_mqtt_client_subscribe(client, "sienovo/boats/BOAT-A3F8/control", 0);
            break;
        case MQTT_EVENT_DATA:
            handle_control_command(evt->data, evt->data_len); // your impl
            break;
        default: break;
    }
}

void mqtt_app_start(void) {
    esp_mqtt_client_config_t cfg = {
        .broker = {
            .address = { .uri = "mqtts://a36ytt8gq852xf-ats.iot.ap-east-1.amazonaws.com:8883" },
            .verification = { .certificate = (const char*)root_ca_pem_start },
        },
        .credentials = {
            .client_id = THING_NAME,
            .authentication = {
                .certificate = (const char*)cert_pem_start,
                .key = (const char*)key_pem_start,
            },
        },
        .session = { .keepalive = 60 },
    };
    client = esp_mqtt_client_init(&cfg);
    esp_mqtt_client_register_event(client, ESP_EVENT_ANY_ID, on_event, NULL);
}
```

```

    esp_mqtt_client_start(client);
}

void publish_state_5hz(void) {
    cJSON* root = cJSON_CreateObject();
    cJSON_AddStringToObject(root, "id", THING_NAME);
    cJSON_AddNumberToObject(root, "lat", get_gps_lat());
    cJSON_AddNumberToObject(root, "lng", get_gps_lng());
    cJSON_AddNumberToObject(root, "heading", get_imu_heading());
    cJSON_AddNumberToObject(root, "speed", get_motor_speed_ms
    ());
    cJSON_AddNumberToObject(root, "battery", get_battery_percent
    ());
    cJSON_AddNumberToObject(root, "signal", get_4g_signal());
    cJSON_AddNumberToObject(root, "baitLevel", get_bait_level());
    cJSON_AddNumberToObject(root, "distance", calc_distance_to_ho
    me());
    cJSON_AddNumberToObject(root, "depth", get_depth_sonar());
    cJSON_AddNumberToObject(root, "waterTemp", get_water_temp());
    cJSON_AddBoolToObject (root, "ir", get_ir_state());
    cJSON_AddBoolToObject (root, "light", get_light_state());
    cJSON_AddBoolToObject (root, "isOnline", true);
    cJSON_AddItemToObject (root, "components", build_components_
    json());
    char* json = cJSON_PrintUnformatted(root);
    esp_mqtt_client_publish(client, "sienovo/boats/B0AT-A3F8/stat
    e", json, 0, 0, 0);
    cJSON_free(json);
    cJSON_Delete(root);
}

```

同等代码用 Arduino + `arduino-mqtt` / `PubSubClient` 也都行。原理都一样：mTLS + JSON。

树莓派 / Linux 端用 Python + `paho-mqtt` 是最快的：~50 行代码就能跑起来（直接照 `simulator/boat.mjs` 翻成 Python）。

9. 常见坑

现象	排查
连接失败 <code>bad certificate</code> / TLS handshake 失败	NTP 没同步、证书放错位置、root CA 没设、cert/key 不配对
连上瞬间被踢 (broker 立即断开)	client ID $\neq$ thing name; 或 IoT policy 没 attach 到这个 cert
<code>subscribe</code> 看似成功但收不到 control 消息	你订阅的 topic 写错了 (注意 <code>+</code> 不能用, 要写 thing 名); 或 Cognito 那端 publish 用了通配符权限不足 (这是 dashboard 那边的事, 找云端)
偶尔丢消息	QoS 0 是正常现象; 上 QoS 1 会缓解, 但占带宽
控制延迟高	可能是你 publish state 太频繁堵塞了出向; 可以暂时把 STATE_HZ 降到 2-3
浏览器看到一半画面	camera frame 太大 (>128KB) 会被 broker 截断; 检查 JPEG 质量和分辨率

10. 不在你范围内的事

不需要你处理:

- ✗ broker 部署 / 维护 — 是 AWS 托管的
- ✗ 用户认证 / 权限 — 是 dashboard 那边走 Cognito + Auth0
- ✗ 数据持久化 / 历史轨迹 — v1 没做, v2 会加 IoT Rule $\rightarrow$ DynamoDB
- ✗ 多设备配对 / 房间逻辑 — broker 自己管 fanout

11. 联络 / 回报

- 协议变更: 本文 markdown 在仓库 [docs/hardware-integration.md](#), 云端有改动会先 PR 到这里再知会
- 接入问题: 先用 `simulator/boat.mjs` 跑通 $\rightarrow$ 再对比你的实现差在哪
- 想加新 topic / 新 message type: 先开 issue 讨论, 不要私下扩协议

